
Principles of Parallel Algorithm Design: Concurrency and Decomposition

Milind Chabbi

Nikhil Hegde

Department of Computer Science and Engineering

IIT Dharwad

Parallel Algorithm

Recipe to solve a problem using multiple processors

Typical steps for constructing a parallel algorithm

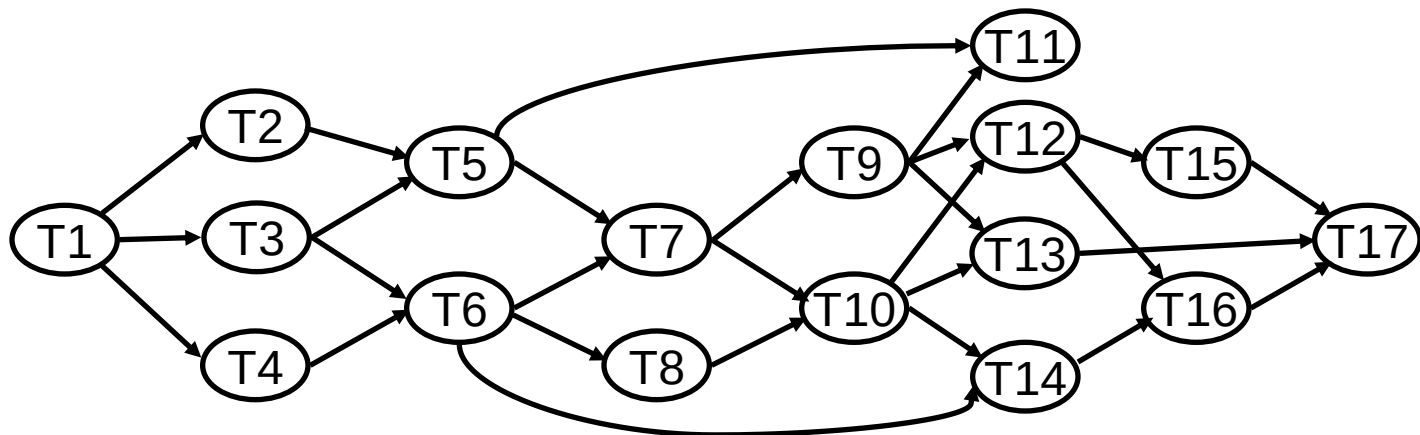
- identify what pieces of work can be performed concurrently
- partition and map work onto independent processors
- distribute a program's input, output, and intermediate data
- coordinate accesses to shared data: avoid conflicts
- ensure proper order of work using synchronization

Why “typical”? Some of the steps may be omitted.

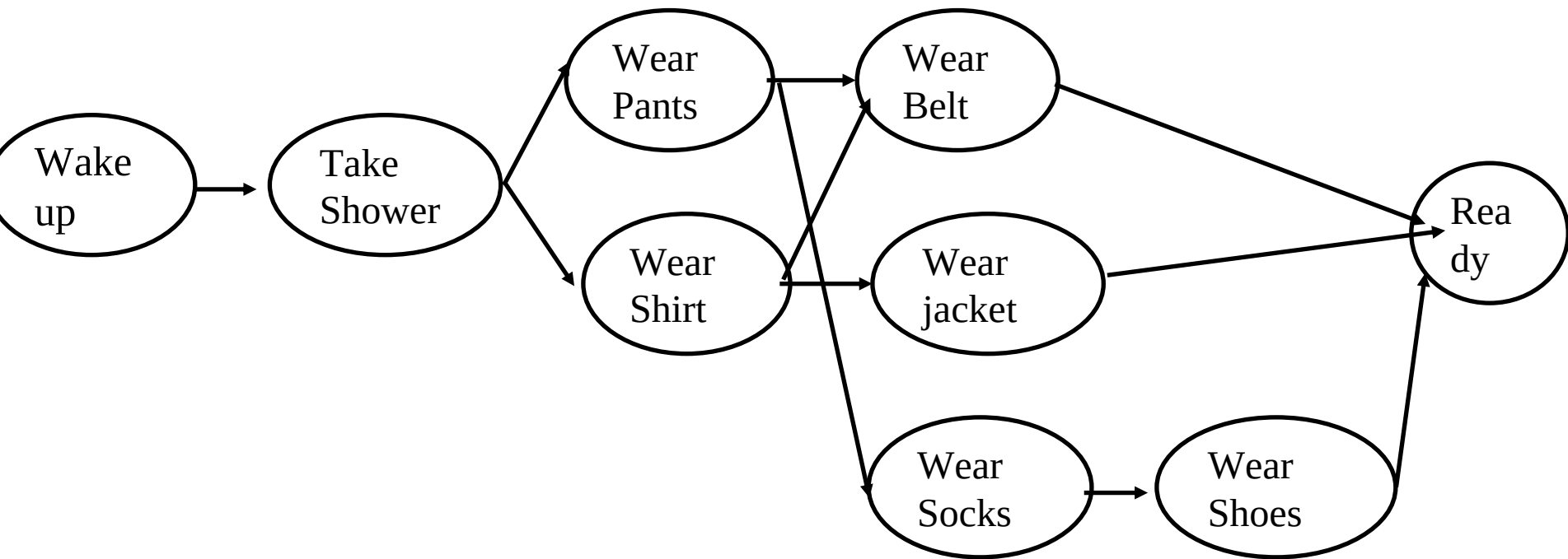
- if data is in shared memory, distributing it may be unnecessary
- if using message passing, there may not be shared data
- the mapping of work to processors can be done statically by the programmer or dynamically by the runtime

Decomposing Work for Parallel Execution

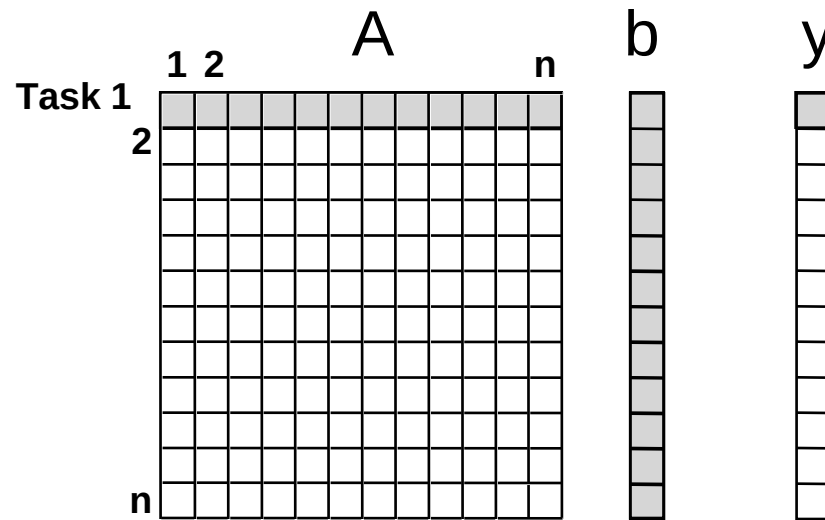
- Divide work into tasks that can be executed concurrently
- Many different decompositions possible for any computation
- Tasks may be same, different, or even indeterminate sizes
- Tasks may be independent or have non-trivial order
- Conceptualize tasks and ordering as task dependency DAG
 - node = task
 - edge = control dependence



Task Dependency Graph



Example: Dense Matrix-Vector Product



- Computing each element of output vector y is independent
- Easy to decompose dense matrix-vector product into tasks
 - one per element in y
- Observations
 - task size is uniform
 - no control dependences between tasks
 - tasks share b

Example: Database Query Processing

Consider the execution of the query:

MODEL = "CIVIC" AND YEAR = 2001 AND

(COLOR = "GREEN" OR COLOR = "WHITE")

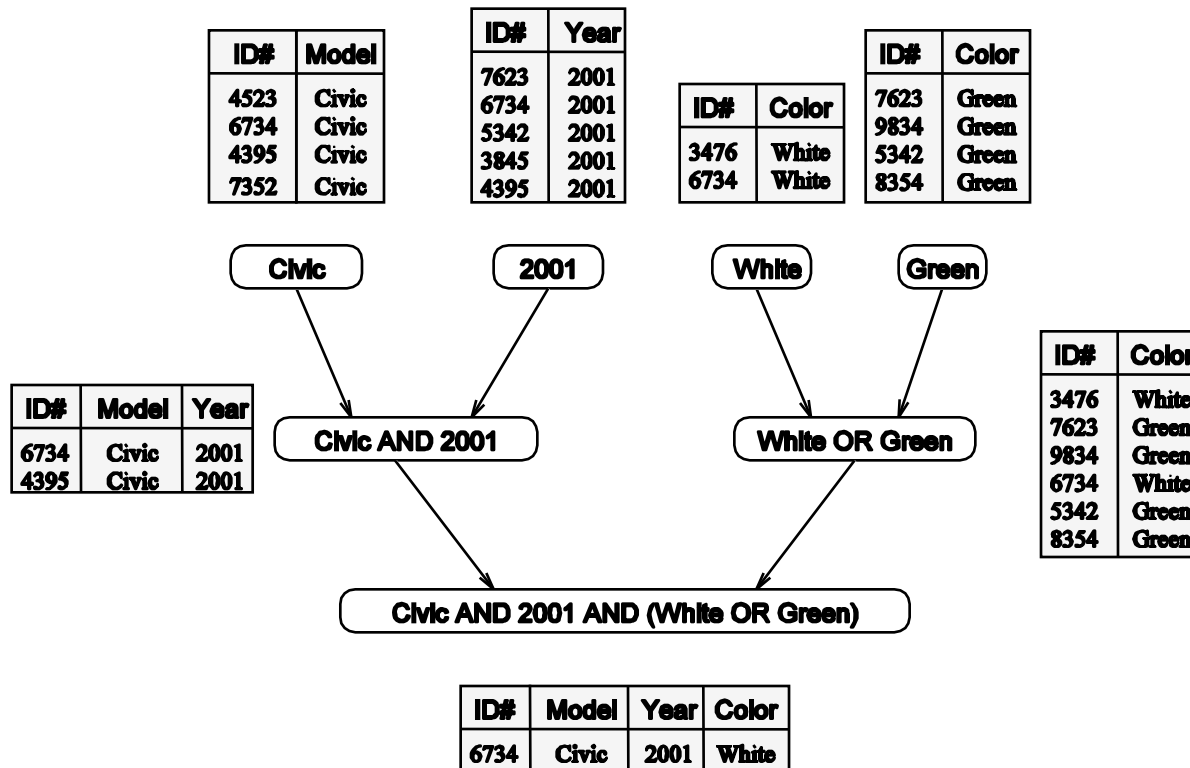
on the following database:

ID#	Model	Year	Color	Dealer	Price
4523	Civic	2002	Blue	MN	\$18,000
3476	Corolla	1999	White	IL	\$15,000
7623	Camry	2001	Green	NY	\$21,000
9834	Prius	2001	Green	CA	\$18,000
6734	Civic	2001	White	OR	\$17,000
5342	Altima	2001	Green	FL	\$19,000
3845	Maxima	2001	Blue	NY	\$22,000
8354	Accord	2000	Green	VT	\$18,000
4395	Civic	2001	Red	CA	\$17,000
7352	Civic	2002	Red	WA	\$18,000

Example: Database Query Processing

- Task: compute set of elements that satisfy a predicate
—task result = table of entries that satisfy the predicate
- Edge: output of one task serves as input to the next

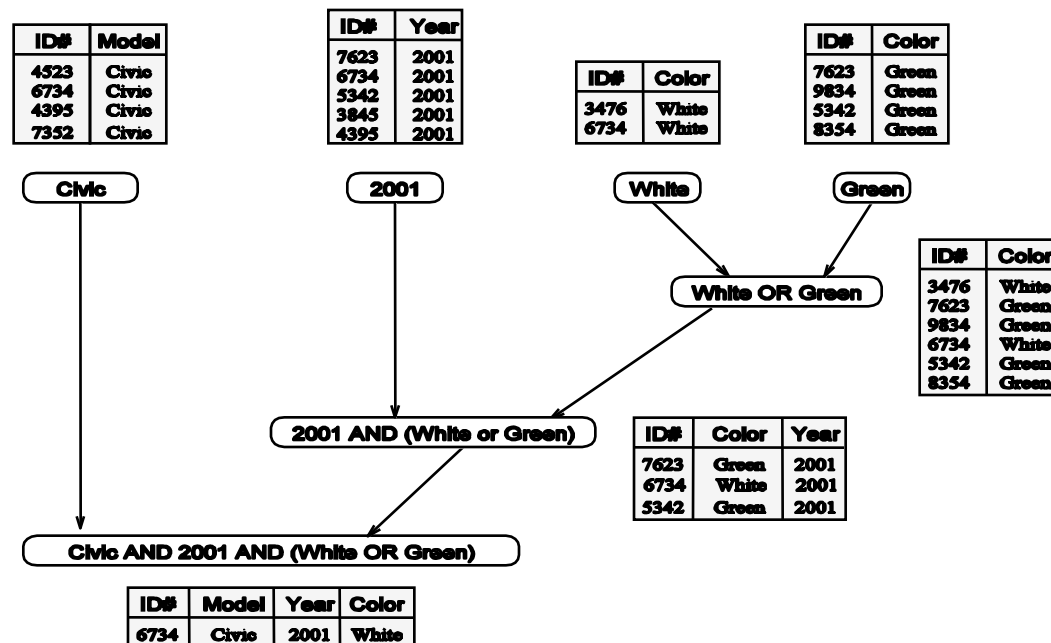
**MODEL = "CIVIC" AND YEAR = 2001 AND
(COLOR = "GREEN" OR COLOR = "WHITE")**



Example: Database Query Processing

- Alternate task decomposition for query

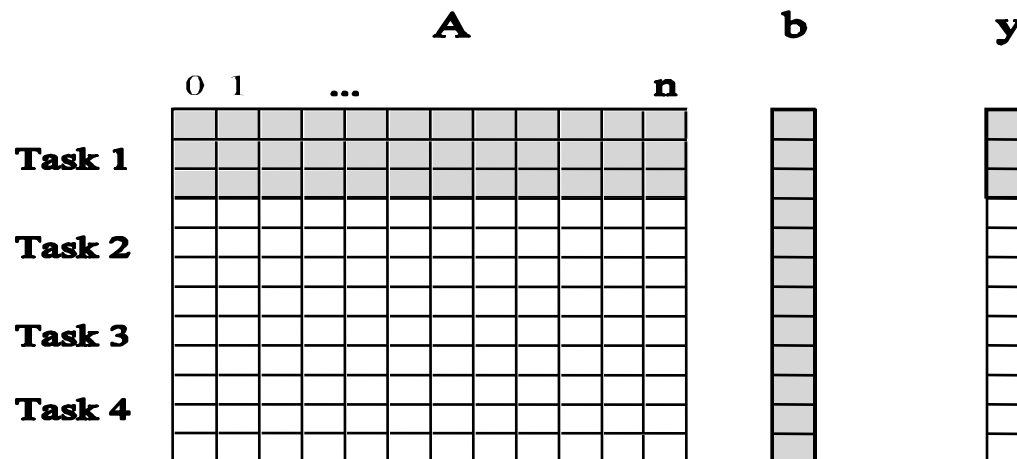
**MODEL = "CIVIC" AND YEAR = 2001 AND
(COLOR = "GREEN" OR COLOR = "WHITE")**



Note: Different decompositions may yield different parallelism and different amounts of work

Granularity of Task Decompositions

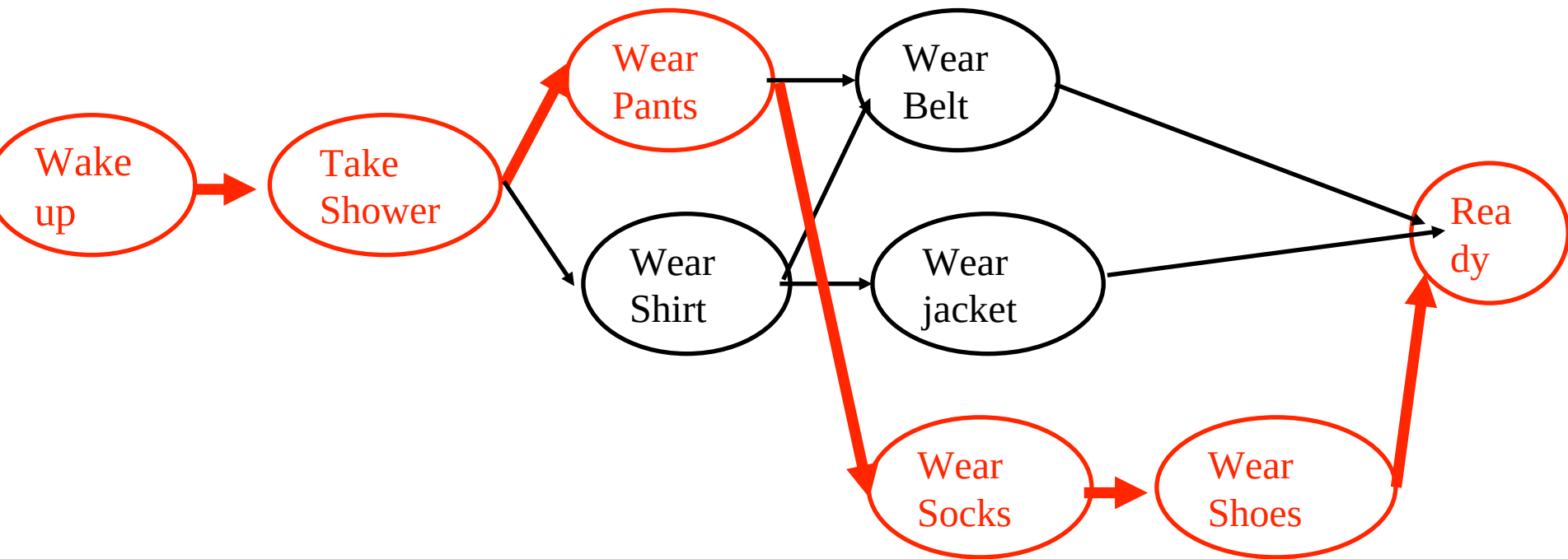
- Granularity = task size
 - depends on the number of tasks
- Fine-grain = large number of tasks
- Coarse-grain = small number of tasks
- Granularity examples for dense matrix-vector multiply
 - fine-grain: each task represents an individual element in y
 - coarser-grain: each task computes 3 elements in y



Critical Path

- **Edge in task dependency graph represents task serialization**
- **Critical path = longest weighted path through graph**
- **Critical path length = lower bound on parallel execution time**

Task Dependency Graph



Degree of Concurrency

- **Definition:** number of tasks that can execute in parallel
- **May change during program execution**
- **Metrics**
 - maximum degree of concurrency
 - largest # concurrent tasks at any point in the execution
 - average parallelism
 - total amount of work divided by length of the critical path
- **Degree of concurrency vs. task granularity**
 - inverse relationship

Metrics

Degree of concurrency

1

1

2

3

1

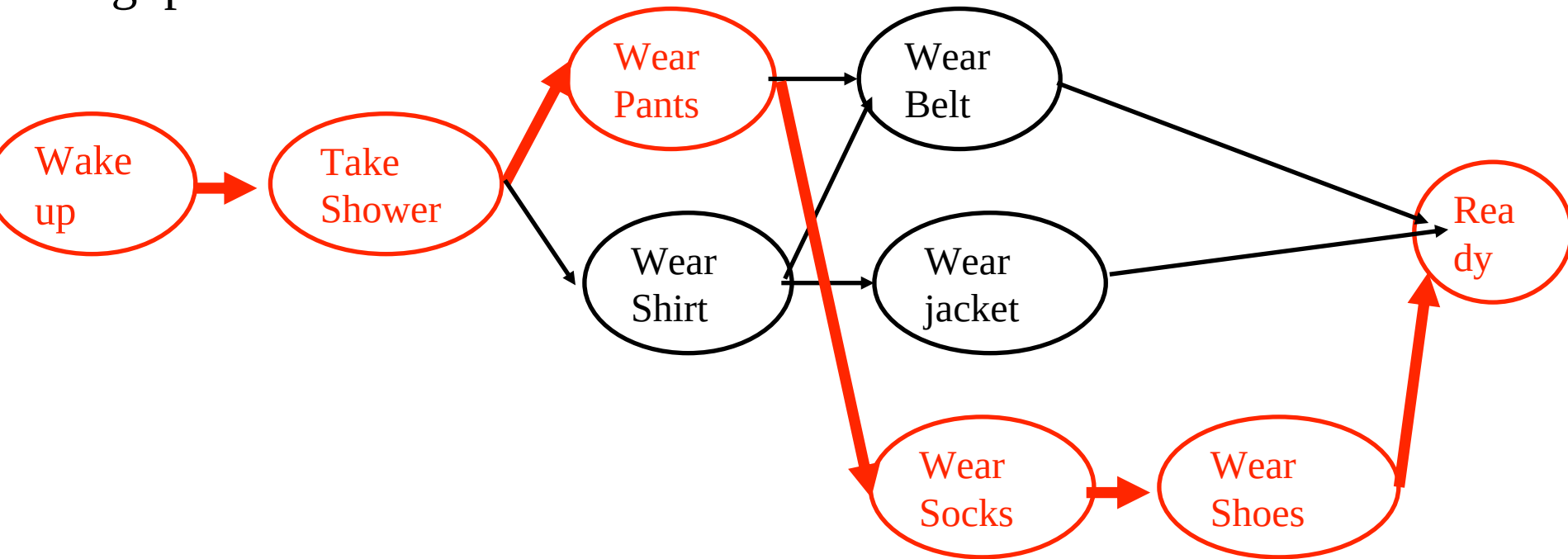
1

Max

Work = 9 (assume unit work / node)

Critical Path Length = 6

Avg. parallelism = $9/6 = 1.5$



Metrics

Degree of concurrency

1

1

2

3

1

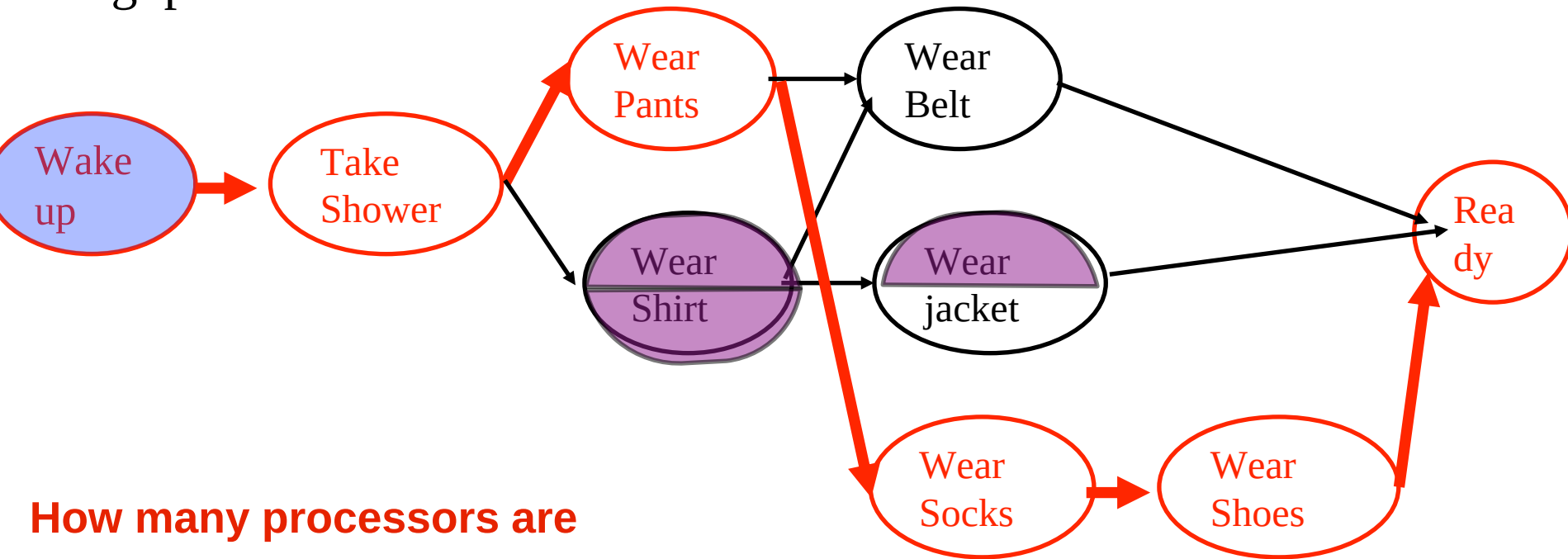
1

Max

Work = 9 (assume unit work / node)

Critical Path Length = 6

Avg. parallelism = $9/6 = 1.5$



**How many processors are
needed to achieve the minimum time?**

Try with 1.5 CPUs

Metrics

Degree of concurrency

1

1

2

3

1

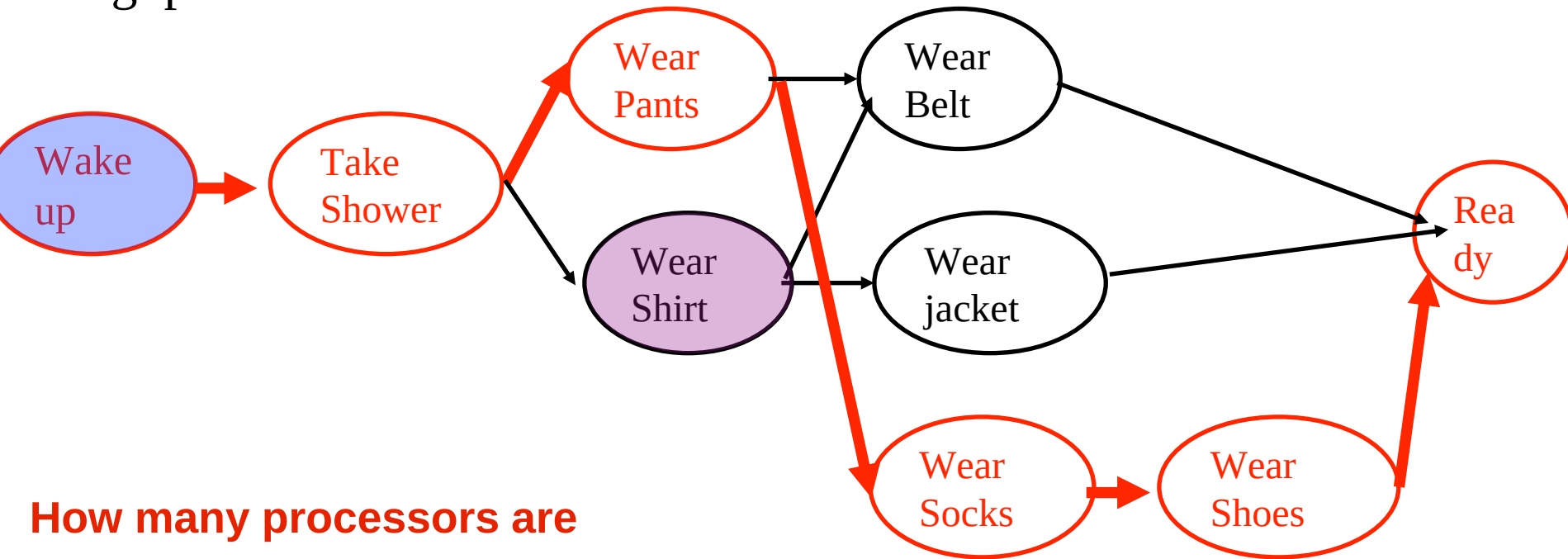
1

Max

Work = 9 (assume unit work / node)

Critical Path Length = 6

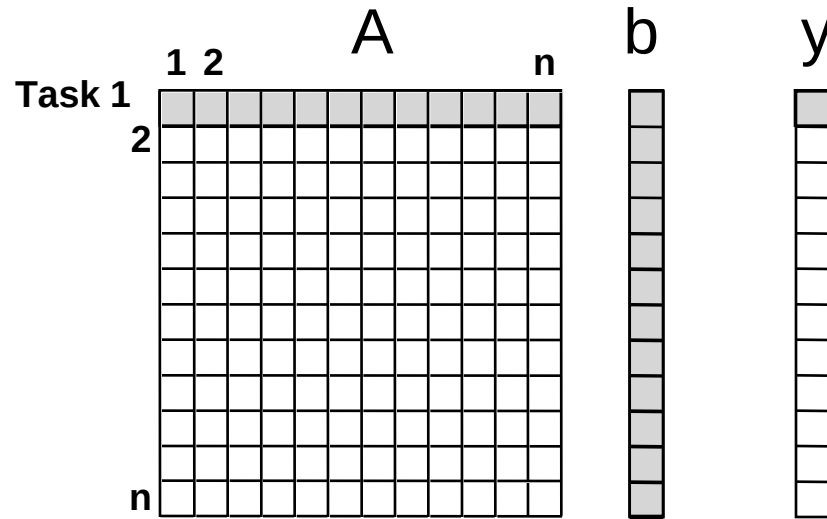
Avg. parallelism = $9/6 = 1.5$



**How many processors are
needed to achieve the minimum time?**

Try with 2 CPUs

Example: Dense Matrix-Vector Multiplication

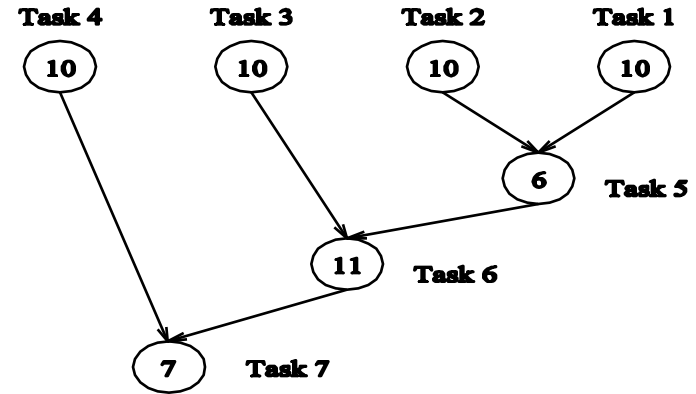
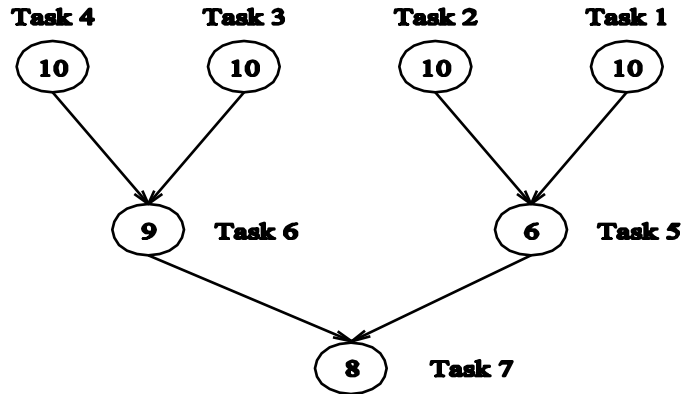


- Computing each element of output vector y is independent
- Easy to decompose dense matrix-vector product into tasks
 - one per element in y
- Observations
 - task size is uniform
 - no control dependences between tasks
 - tasks share b

Question: Is n the maximum number of tasks possible?

Critical Path Length

Examples: database query task dependency graphs



Note: number in each vertex represents the cost of its task

Questions:

What tasks are on the critical path for each dependency graph?

What is the shortest parallel execution time for each decomposition?

What is the maximum degree of concurrency?

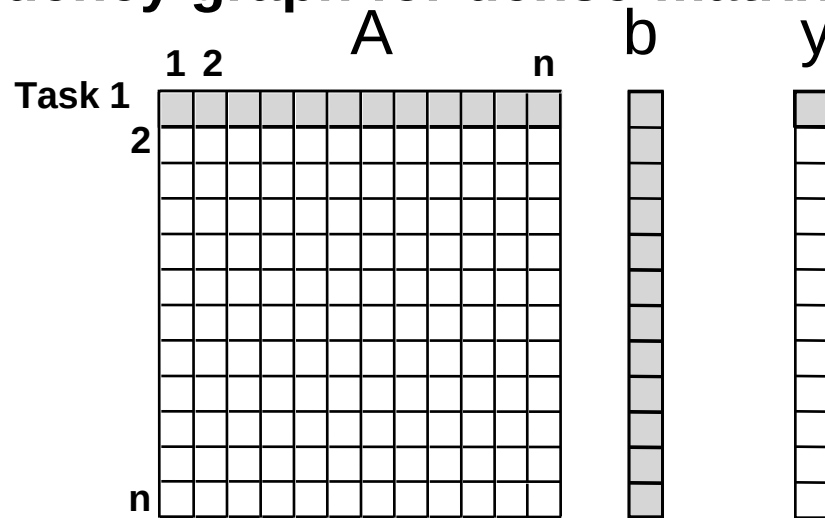
What is the average parallelism?

How many processors are needed to achieve the minimum time?

For simplicity: assume each node takes unit CPU time.

Critical Path Length

Example: dependency graph for dense-matrix vector product



Questions:

What is the maximum number of tasks possible?

What does a task dependency graph look like for this case?

What is the shortest parallel execution time for the graph?

What is the maximum degree of concurrency?

How many processors are needed to achieve the minimum time?

What is the average parallelism?

Computation Graph: Dot Product for One Row

